COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S3
DATE PERFORMED:	SEPTEMBER 10 2026	**DATE SUBMITTED:**	SEPTEMBER 15 2026
NAME:	DUSTIN CARL B. MAPAS	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 8.1 **Hands-on Activity 8.1 Sorting Algorithms Pt2**

1. PROCEDURE OUTPUT

Code + Console Screenshot	**Code:**
	`template <typename T>`
	`void printArray(T arr[], int arrSize) {`
	`  for (int i = 0; i < arrSize; i++) {`
	`  std::cout << arr[i] << (i == arrSize - 1 ? "" : ", ");`
	`  if ((i + 1) % 10 == 0) {`
	`  std::cout << "\n";`
	`  }`
	`  }`
	`  std::cout << "\n";`
	`}`

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "SortingAlgo.h"

const int SIZE = 100;


int main() {
    int unsorted[SIZE];
    int shellArr[SIZE], mergeArr[SIZE], quickArr[SIZE];

    srand( Seed: time(NULL));
    for (int i = 0; i < SIZE; i++) {
        unsorted[i] = rand() % 100;
        shellArr[i] = unsorted[i];
        mergeArr[i] = unsorted[i];
        quickArr[i] = unsorted[i];
    }

    std::cout << "Unsorted" << std::endl;
    printArray(unsorted, SIZE);

```

Output:

```
Unsorted
54, 50, 38, 24, 39, 46, 22, 94, 34, 16,
8, 25, 11, 8, 93, 94, 8, 82, 90, 99,
75, 8, 82, 5, 62, 27, 13, 52, 18, 67,
85, 88, 76, 70, 69, 74, 19, 71, 35, 65,
32, 21, 18, 2, 58, 93, 51, 61, 19, 80,
71, 1, 54, 89, 50, 62, 81, 44, 82, 78,
38, 6, 31, 47, 0, 96, 62, 54, 81, 62,
8, 74, 61, 79, 64, 71, 62, 55, 47, 9,
54, 68, 15, 62, 84, 1, 66, 90, 74, 68,
60, 72, 31, 65, 91, 58, 76, 55, 95, 96
```

Observations	The output displays an unsorted array because of a random seed number generator, and by

	using a for loop, I stored that number in an array, and then printed it using my printing function.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	**Code:**
	`template <typename T>`
	`void shellSort(T arr[], int arrSize) {`
	`  for (int gap = arrSize / 2; gap > 0; gap /= 2) {`
	`  for (int i = gap; i < arrSize; i++) {`
	`  T temp = arr[i];`
	`  int j = i;`
	`  while (j >= gap && arr[j - gap] > temp) {`
	`  arr[j] = arr[j - gap];`
	`  j -= gap;`
	`  }`
	`  arr[j] = temp;`
	`  }`
	`  }`
	`}`
	`  std::cout << "Testing Shell Sort" << std::endl;`
	`  shellSort(shellArr, SIZE);`
	`  printArray(shellArr, SIZE);`
	Output:
	`Testing Shell Sort`
	`0, 1, 1, 2, 5, 6, 8, 8, 8, 8,`
	`8, 9, 11, 13, 15, 16, 18, 18, 19, 19,`
	`21, 22, 24, 25, 27, 31, 31, 32, 34, 35,`
	`38, 38, 39, 44, 46, 47, 47, 50, 50, 51,`
	`52, 54, 54, 54, 54, 55, 55, 58, 58, 60,`
	`61, 61, 62, 62, 62, 62, 62, 62, 64, 65,`
	`65, 66, 67, 68, 68, 69, 70, 71, 71, 71,`
	`72, 74, 74, 74, 75, 76, 76, 78, 79, 80,`
	`81, 81, 82, 82, 82, 84, 85, 88, 89, 90,`
	`90, 91, 93, 93, 94, 94, 95, 96, 96, 99`

Observations	Shell short is like an enhanced version of insertion sort. Thus sorting the elements apart by using gaps that shrink. This helps the sorting algorithm to reduce the total number of swaps

	required to fully order the unsorted array. The worst case time complexity of this is O(n^2), while the best case is O(n log n).

Table 8-2. Shell Sort Technique

Code + Console Screenshot	**Code:**
	23 template <typename T>
	24 void merge(T arr[], int left, int middle, int right) {
	25 int n1 = middle - left + 1;
	26 int n2 = right - middle;
	27
	28 T* L = new T[n1];
	29 T* R = new T[n2];
	30
	31 for (int i = 0; i < n1; i++) {
	32 L[i] = arr[left + i];
	33 }
	34 for (int j = 0; j < n2; j++) {
	35 R[j] = arr[middle + 1 + j];
	36 }
	37
	38 int i = 0, j = 0, k = left;
	39 while (i < n1 && j < n2) {
	40 if (L[i] <= R[j]) {
	41 arr[k] = L[i];
	42 i++;
	43 } else {
	44 arr[k] = R[j];
	45 j++;
	46 }
	47 k++;
	48 }
	49
	50 while (i < n1) {
	51 arr[k] = L[i];
	52 i++;
	53 k++;
	54 }
	55
	56 while (j < n2) {
	57 arr[k] = R[j];

```
            arr[k] = R[j];
            j++;
            k++;
        }

        delete[] L;
        delete[] R;
    }

    template <typename T>
    void mergeSort(T arr[], int left, int right) {
        if (left < right) {
            int middle = (left + right) / 2;
            mergeSort(arr, left, right: middle);
            mergeSort(arr, left: middle + 1, right);
            merge(arr, left, middle, right);
        }
    }
```

```
    std::cout << "Testing Merge Sort" << std::endl;
    mergeSort(mergeArr, left: 0, right: SIZE - 1);
    printArray(mergeArr, SIZE);
```

Output:

```
Testing Merge Sort
0, 1, 1, 2, 5, 6, 8, 8, 8, 8,
8, 9, 11, 13, 15, 16, 18, 18, 19, 19,
21, 22, 24, 25, 27, 31, 31, 32, 34, 35,
38, 38, 39, 44, 46, 47, 47, 50, 50, 51,
52, 54, 54, 54, 54, 55, 55, 58, 58, 60,
61, 61, 62, 62, 62, 62, 62, 62, 64, 65,
65, 66, 67, 68, 68, 69, 70, 71, 71, 71,
72, 74, 74, 74, 75, 76, 76, 78, 79, 80,
81, 81, 82, 82, 82, 84, 85, 88, 89, 90,
90, 91, 93, 93, 94, 94, 95, 96, 96, 99
```

Observations	A merge sort technique breaks down an array into individual sub arrays before rejoining them in a sorted order. This sorting technique uses the divide and conquer strategy and the time

	complexity of this is O(n log n).

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	**Code:**
	77 template <typename T>
	78 int partition(T arr[], int low, int high) {
	79 T pivot = arr[high]; // Last element as pivot
	80 int i = low - 1;
	81
	82 for (int j = low; j < high; j++) {
	83 if (arr[j] < pivot) {
	84 i++;
	85 std::swap([&] arr[i], [&] arr[j]);
	86 }
	87 }
	88 std::swap([&] arr[i + 1], [&] arr[high]);
	89 return i + 1;
	90 }
	91
	92 template <typename T>
	93 void quickSort(T arr[], int low, int high) {
	94 if (low < high) {
	95 int pivot = partition(arr, low, high);
	96 quickSort(arr, low, high: pivot - 1);
	97 quickSort(arr, low: pivot + 1, high);
	98 }
	99 }
	2 std::cout << "Testing Quick Sort" << std::endl;
	3 quickSort(quickArr, low: 0, high: SIZE - 1);
	4 printArray(quickArr, SIZE);
	5
	Output:

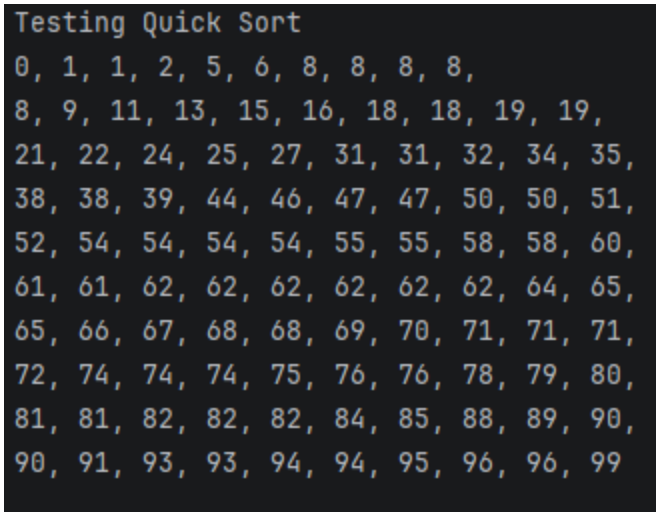
	
Observations	Quick sort partitions the array around the chosen pivot element. In this case the last element is the pivot. Furthermore the smaller elements are moved to the left while the larger elements are moved to the right on the pivot. The worst case time complexity of this is O(n^2), while the best case is O(n log n).

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

- Yes we can sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms like insertion sort..

Code:

```cpp
#ifndef SUPPHEADER_H
#define SUPPHEADER_H

template <typename T>
void insertionSort(T arr[], int low, int high) {
    for (int i = low + 1; i <= high; i++) {
        T temp = arr[i];
        int j = i - 1;
        while (j >= low && arr[j] > temp) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = temp;
    }
}

template <typename T>
void hybridQuickSort(T arr[], int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
        insertionSort(arr, low, high: pivotIndex - 1);
        insertionSort(arr, low: pivotIndex + 1, high);
    }
}

#endif #ifndef SUPPHEADER_H
```

```cpp
#include <iostream>
#include "SortingAlgo.h"
#include "suppheader.h"

int main() {
    int arr[] = {48, 21, 10, 15, 57, 29};
    int size = sizeof(arr) / sizeof(arr[0]);

    std::cout << "Original Array: ";
    printArray(arr, size);

    //partitioning
    int pivotIdx = partition(arr, low: 0, high: size - 1);

    std::cout << "After Partition (Pivot = " << arr[pivotIdx];
    std::cout << " at index " << pivotIdx << "): ";
    printArray(arr, size);

    //sorting left sub list using insertion sort
    insertionSort(arr, low: 0, high: pivotIdx - 1);

    //sorting right sub list using insertion sort
    insertionSort(arr, low: pivotIdx + 1, high: size - 1);

    std::cout << "Final Fully Sorted Array: ";
    printArray(arr, size);

    return 0;
}
```

Output:

```
Original Array: 48, 21, 10, 15, 57, 29
After Partition (Pivot = 29 at index 3): 21, 10, 15, 29, 57, 48
Final Fully Sorted Array: 10, 15, 21, 29, 48, 57

Process finished with exit code 0
```

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

- Quick sort will give the fastest time performance because it operates entirely in place which means there will be minimal memory overhead. Furthermore, merge sort and quick sort have O(N • log N) for their time complexity because divide and conquer strategy recursively splits the array into a tree of log2 N levels. And then comparing and sorting those elements across all sub list requires linear work thus resulting to O(N • log N).

Conclusion:
After doing this hands-on activity, I can conclude that this activity helped me to create a C++ code for implementation of shell, merge,and quick sort sorting techniques. Moreover, I was also able to solve given data sorting problems using appropriate sorting algorithms. In addition to this, I also did my observations for the procedure part and for the supplementary part I completed every activity. Lastly, I will keep on learning to enhance my computer programming skills.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts, and code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used gemini as a tool for understanding and to guide me in coding.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.